



Real-time Attendance System

Name: Phoong Guik Ming

StudentID: 2006616



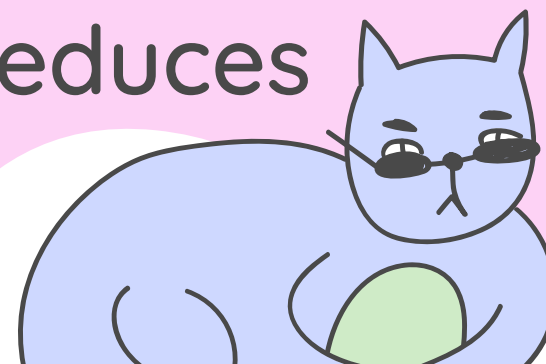
Introduction

Introduction



- Purpose: Manage and track student attendance
- Problem: Manual attendance is time-consuming and error-prone.
- Solution: Automated system using facial recognition and motion sensors to track attendance.
- Objectives: Build low-cost implementation, reduced resource consumption, contactless recognition, real-time monitoring and prevent cheating, reduces manual efforts

20



Name: Phoong Guik Ming. Student id: 2006616

Programme: Computer science

Real-time Attendance System



INTRODUCTION

- Problem: Manual attendance is time-consuming and error-prone.
- Solution: Automated system using facial recognition and motion sensors to track attendance.

SYSTEM ADVANTAGES

- Low cost: Utilizes affordable hardware
- Contactless: No physical contact required.
- Real-Time Monitoring: Instant attendance updates.
- Reduced Effort: Minimizes manual attendance tracking

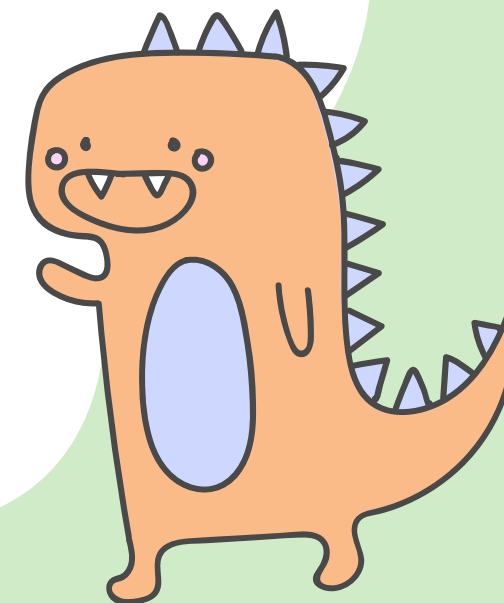
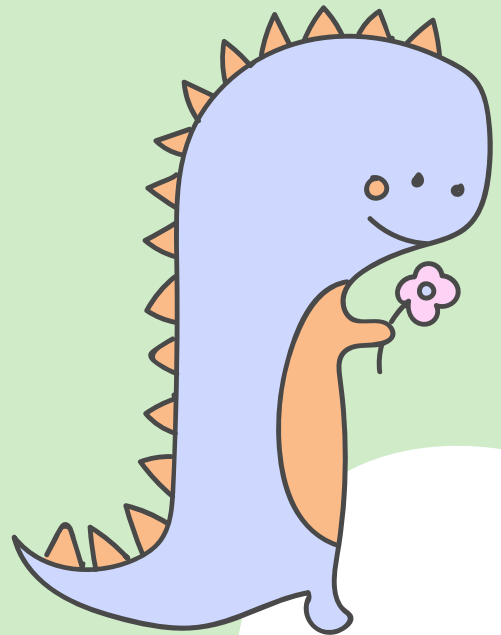
KEY FEATURES:

- Automated Attendance: No manual roll call needed.
- Facial Recognition: Uses OpenCV and Dlib for accurate identification.
- Real-Time Monitoring: Alerts for low or irregular attendance.



Hardware: Raspberry Pi 5, LED, Ultrasonic Sensor, PIR Sensor, Webcam

Literature Review

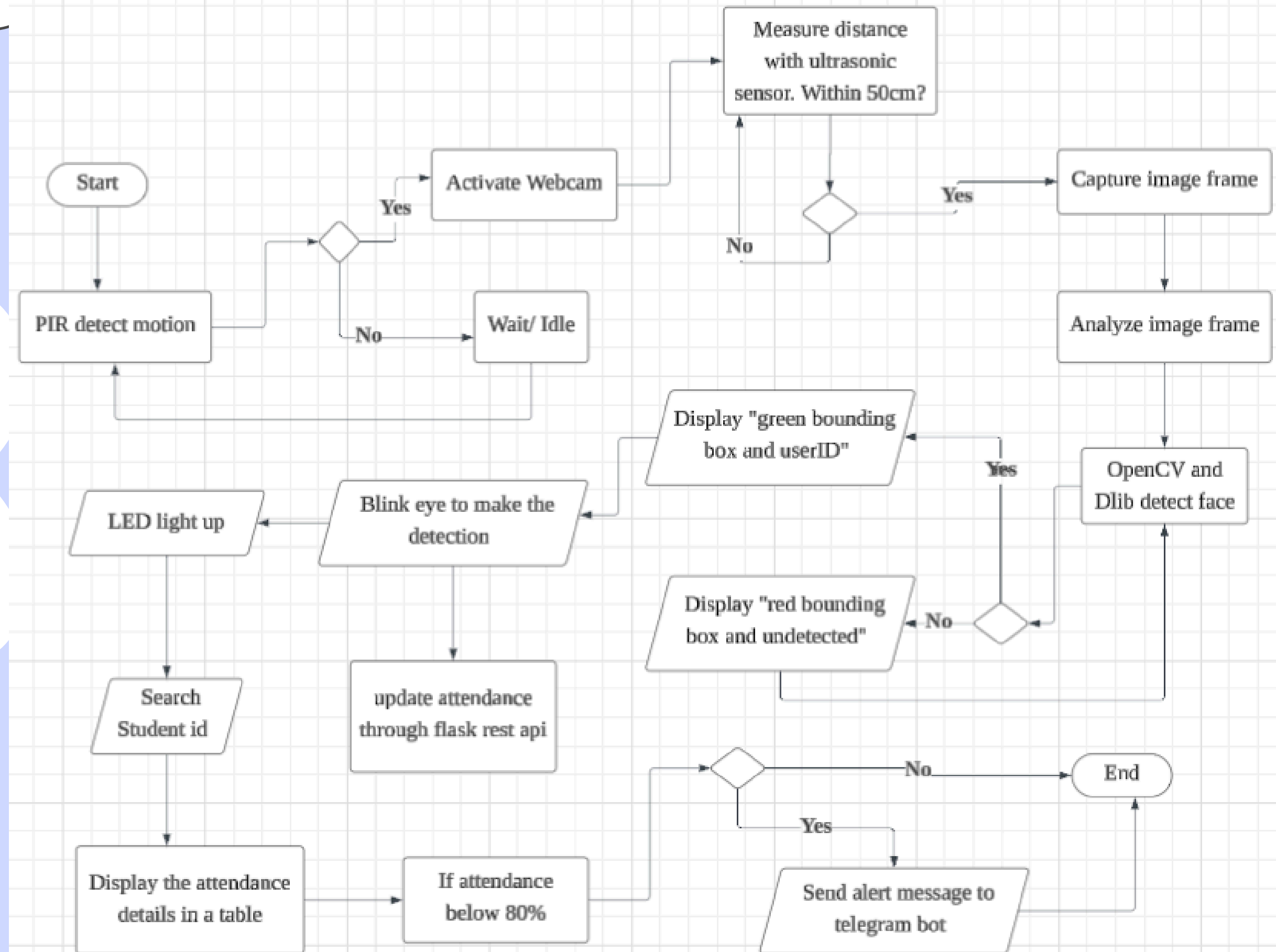


Review of the Technologies

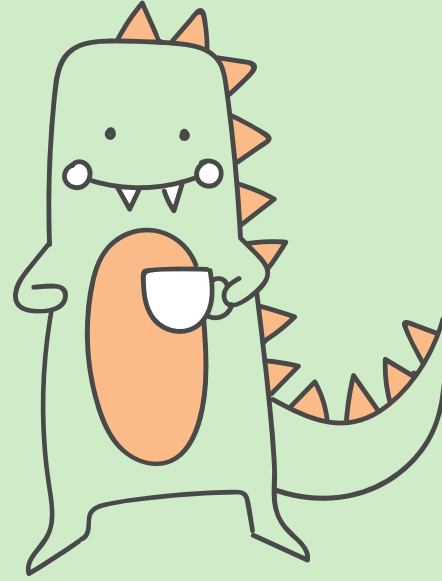
- Hardware: Raspberry Pi 5 (BCM2712), 32GB microSD, LPDDR4X-4267 SDRAM, PIR sensor, HD webcam, ultrasonic sensor, LED.
- Firmware/OS: Debian Bookworm (Linux-based).
- Database: MySQL or SQLite
- Programming language: Python (with OpenCV, Dlib, MySQL Connector/SQLite3) and HTML for the web interface.
- Face detection and recognition (OpenCV, Dlib), attendance logging with PIR and ultrasonic sensor for accuracy

Methodology

System Flowchart



System Implementation

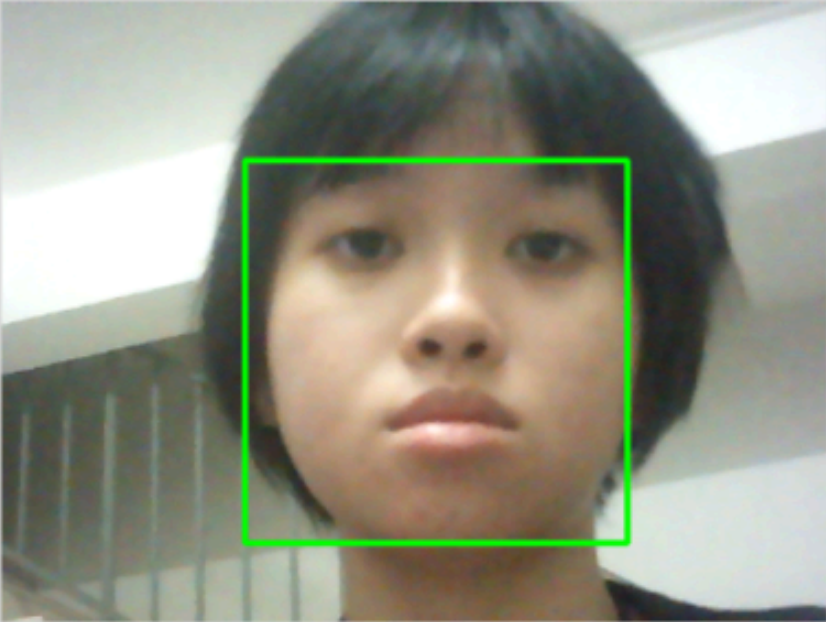


Register

```
yueming@raspberrypi: ~/New/face1-test
File Edit Tabs Help
[DEBUG] EAR: 0.320
[DEBUG] PIR value: 1
[DEBUG] Measured distance: 16.33 cm
[DEBUG] Camera read success: True
[INFO] No active course at 2025-09-23 05:44:04.
[DEBUG] EAR: 0.324
[DEBUG] PIR value: 1
[DEBUG] Measured distance: 16.32 cm
[DEBUG] Camera read success: True
[INFO] No active course at 2025-09-23 05:44:04.
[DEBUG] EAR: 0.085
[DEBUG] Blink detected, freezing frame.
GPIO released. Program stopped.
(myenvvv) yueming@raspberrypi:~/New/face1-test $ python reg1.py
Photo saved: ./data/photos/S002.jpg
Error during API request: HTTPConnectionPool(host='127.0.0.1', port=5000): Max r
etries exceeded with url: /api/users/ (Caused by NewConnectionError('<urllib3.co
nnection.HTTPConnection object at 0x7ffefc99de10>: Failed to establish a new con
nection: [Errno 111] Connection refused'))
(myenvvv) yueming@raspberrypi:~/New/face1-test $ python reg1.py
Photo saved: ./data/photos/S003.jpg
User ming saved successfully!
User data saved locally to ./user_info.json
█
```

Register User Information

Photo Taken, Now Enter User Information



Take Photo

Name

ming

Email

ming111@gmail.com

Faculty

fict

Student ID

2006616

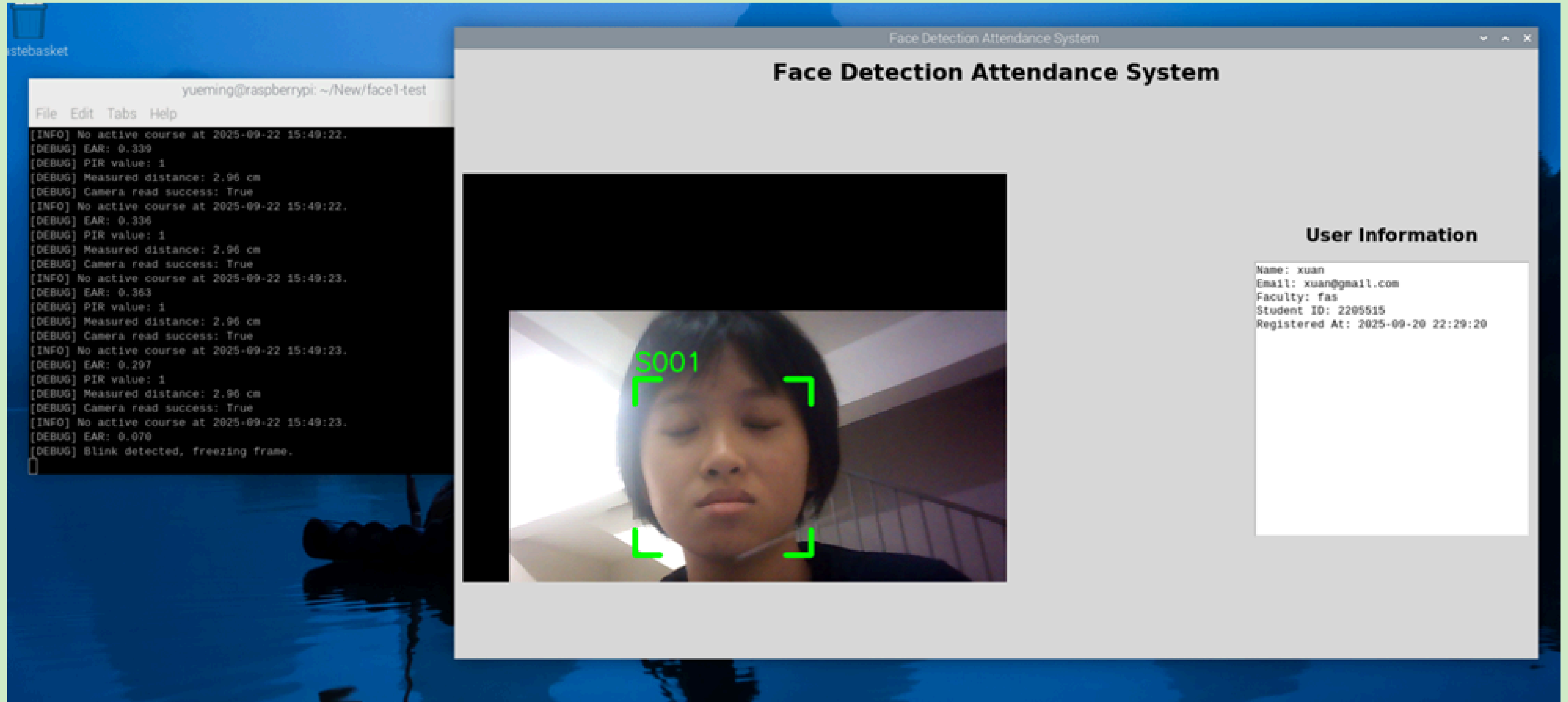
Save

Update User Information through API

```
yueming@raspberrypi: ~/New/face1-test
File Edit Tabs Help
yueming@raspberrypi:~$ cd /home/yueming/New/face1-test
yueming@raspberrypi:~/New/face1-test$ source myenvv/bin/activate
(myenvv) yueming@raspberrypi:~/New/face1-test$ python api1.py
* Serving Flask app 'api1'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment
Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [23/Sep/2025 05:47:18] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [23/Sep/2025 05:47:19] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [23/Sep/2025 05:47:24] "GET /api/users/ HTTP/1.1" 200 -
^[[A^[[B127.0.0.1 - - [23/Sep/2025 05:48:45] "POST /api/users/ HTTP/1.1" 201 -
127.0.0.1 - - [23/Sep/2025 05:49:12] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [23/Sep/2025 05:49:12] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [23/Sep/2025 05:49:15] "GET /api/users/ HTTP/1.1" 200 -
[]
```

```
127.0.0.1:5000/api/users/ - Chromium
127.0.0.1:5000/api/users/
Pretty-print ☐
[
  {
    "id": 13,
    "photo_id": "S001",
    "name": "xuan",
    "email": "xuan@gmail.com",
    "faculty": "fas",
    "student_id": "2205515",
    "register_datetime": "2025-09-20 14:29:20"
  },
  {
    "id": 14,
    "photo_id": "S003",
    "name": "ming",
    "email": "ming111@gmail.com",
    "faculty": "fict",
    "student_id": "2006616",
    "register_datetime": "2025-09-22 21:48:45"
  }
]
```

Blink Detection trigger with PIR and Ultrasonic Sensor



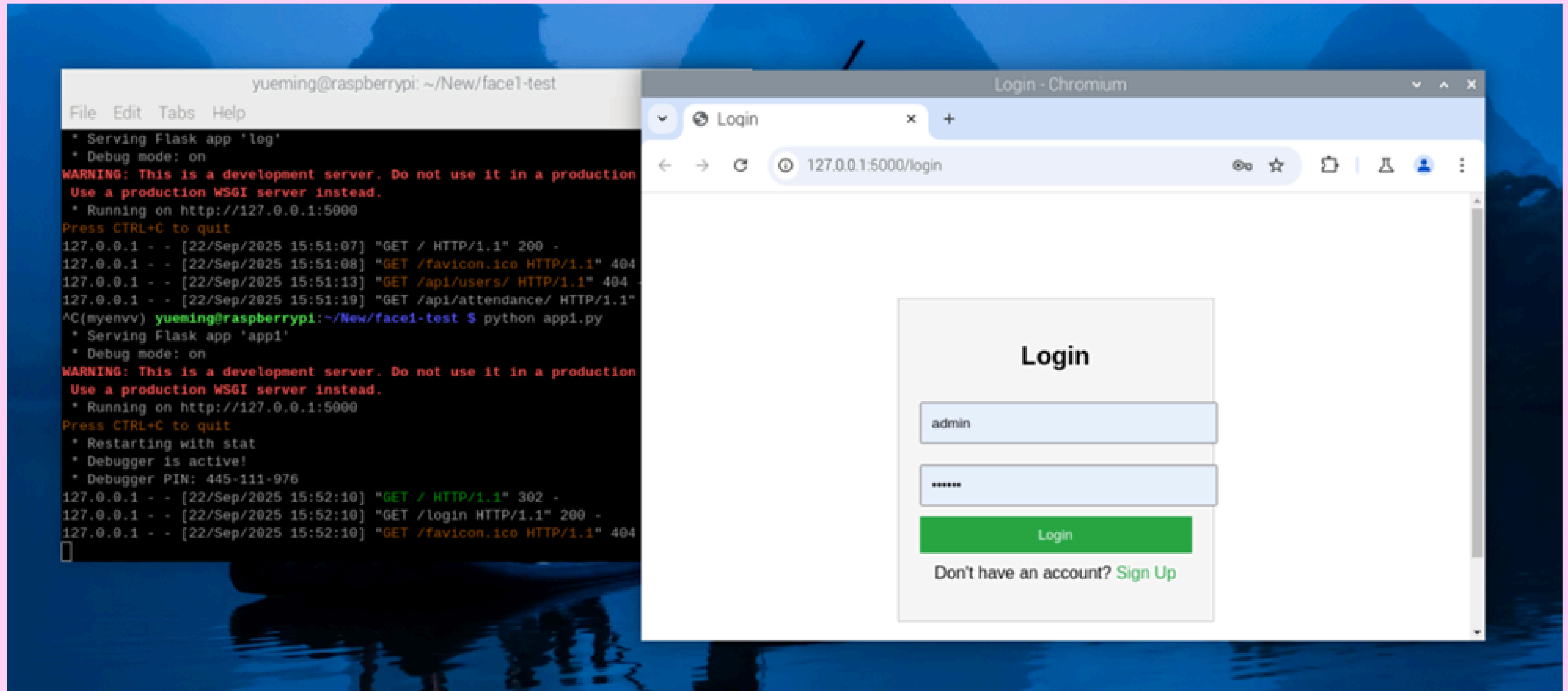
Update User Attendance through API

```
yueming@raspberrypi: ~/New/face1-test
File Edit Tabs Help

yueming@raspberrypi:~ $ cd /home/yueming/New/face1-test
yueming@raspberrypi:~/New/face1-test $ source myenvv/bin/activate
(myenvv) yueming@raspberrypi:~/New/face1-test $ python log.py
* Serving Flask app 'log'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment
Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [22/Sep/2025 15:51:07] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2025 15:51:08] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [22/Sep/2025 15:51:13] "GET /api/users/ HTTP/1.1" 404 -
127.0.0.1 - - [22/Sep/2025 15:51:19] "GET /api/attendance/ HTTP/1.1" 200 -
[]
```

```
127.0.0.1:5000/api/attendance/ - Chromium
127.0.0.1:5000/api/atte x +
127.0.0.1:5000/api/attendance/
Pretty-print ☐
[
  {
    "id": 5,
    "student_id": "2205515",
    "timestamp": "2025-09-21 02:50:10",
    "course_id": "C001",
    "course_name": "Computer Vision"
  },
  {
    "id": 6,
    "student_id": "2205515",
    "timestamp": "2025-09-21 04:14:58",
    "course_id": "C003",
    "course_name": "Database"
  }
]
```

User Login and password hashing



User Attendance Record

```
yueming@raspberrypi: ~/New/face1-test
File Edit Tabs Help
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [22/Sep/2025 15:51:07] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2025 15:51:08] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [22/Sep/2025 15:51:13] "GET /api/users/ HTTP/1.1" 404 -
127.0.0.1 - - [22/Sep/2025 15:51:19] "GET /api/attendance/ HTTP/1.1"
^C(myenvv) yueming@raspberrypi:~/New/face1-test $ python app1.py
* Serving Flask app 'app1'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production
Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 445-111-976
127.0.0.1 - - [22/Sep/2025 15:52:10] "GET / HTTP/1.1" 302 -
127.0.0.1 - - [22/Sep/2025 15:52:10] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2025 15:52:10] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [22/Sep/2025 15:52:45] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2025 15:52:48] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [22/Sep/2025 15:52:48] "GET /index HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2025 15:52:53] "POST /index HTTP/1.1" 200 -
█
```

Attendance Tracker - Chromium

Attendance Tracker

127.0.0.1:5000/index

Logout

Check Student Attendance

Enter Student ID:

Attendance Report for: 2205515

Download CSV

Course ID	Course Name	Time	Total Sessions	Attended	Percentage
C001	Computer Vision	02:00 - 03:00	2	2	100.0%
C002	Machine Learning	01:00 - 02:00	2	0	0.0%
C003	Database	04:00 - 05:00	1	1	100.0%

Alert message

```
yueming@raspberrypi: ~/New/face1-test
File Edit Tabs Help

(yuening@raspberrypi:~/New/face1-test $ python bot2.py
Scheduler started and will check attendance every Monday at 3:59 PM.
Scheduler is running in the background.
^CTraceback (most recent call last):
  File "/home/yuening/New/face1-test/bot2.py", line 138, in <module>
    print("Scheduler is running in the background.")
KeyboardInterrupt

(yuening@raspberrypi:~/New/face1-test $ python bot2.py
Scheduled the job to run on Monday at 4:00 PM.
Scheduler is running in the background.
^CTraceback (most recent call last):
  File "/home/yuening/New/face1-test/bot2.py", line 140, in <module>
    time.sleep(1) # Keep the script running and waiting for scheduled jobs
KeyboardInterrupt

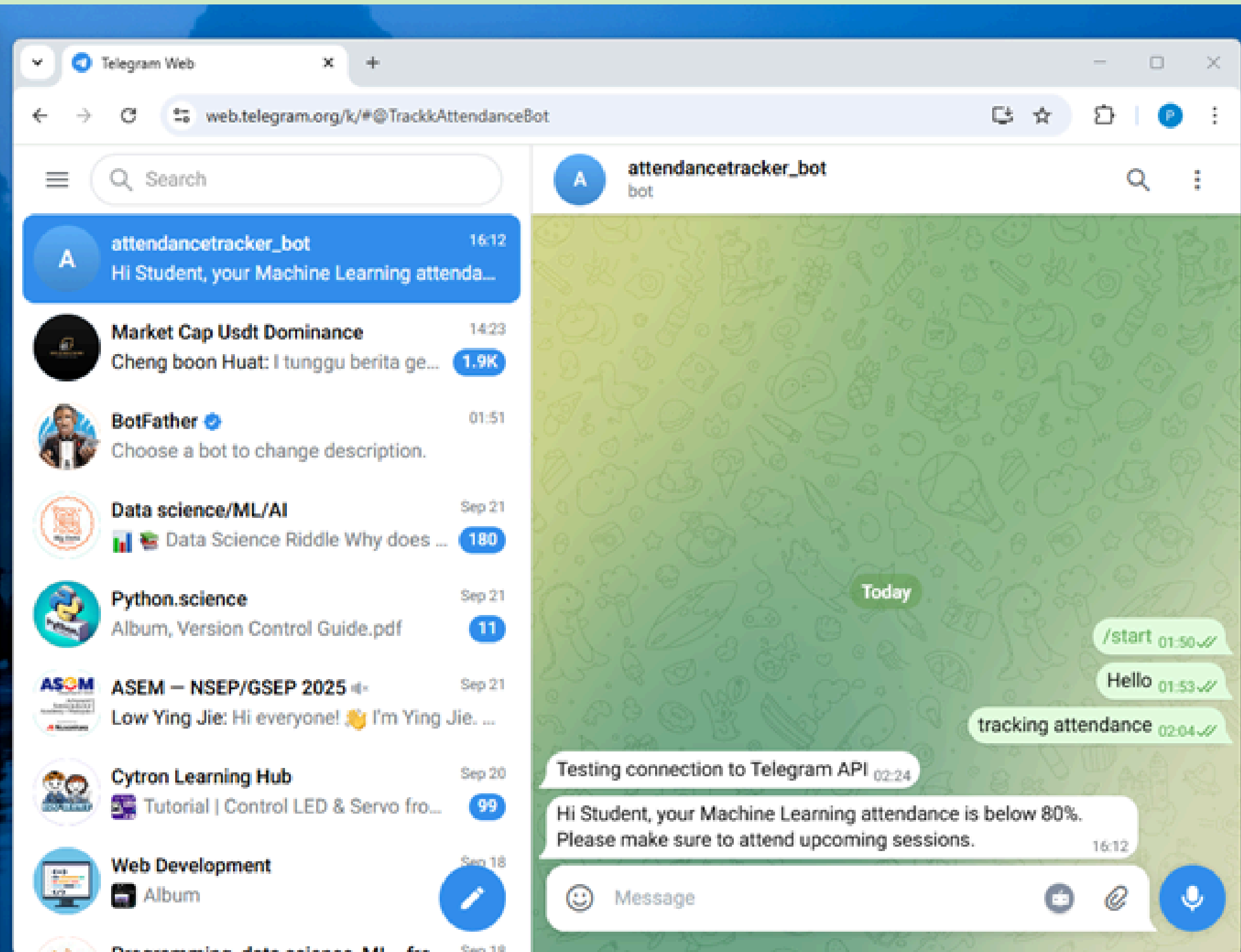
(yuening@raspberrypi:~/New/face1-test $ python bot3.py
Scheduler started and will check attendance every minute.
Scheduler is running in the background.
Checking attendance for all students...
Message sent successfully!

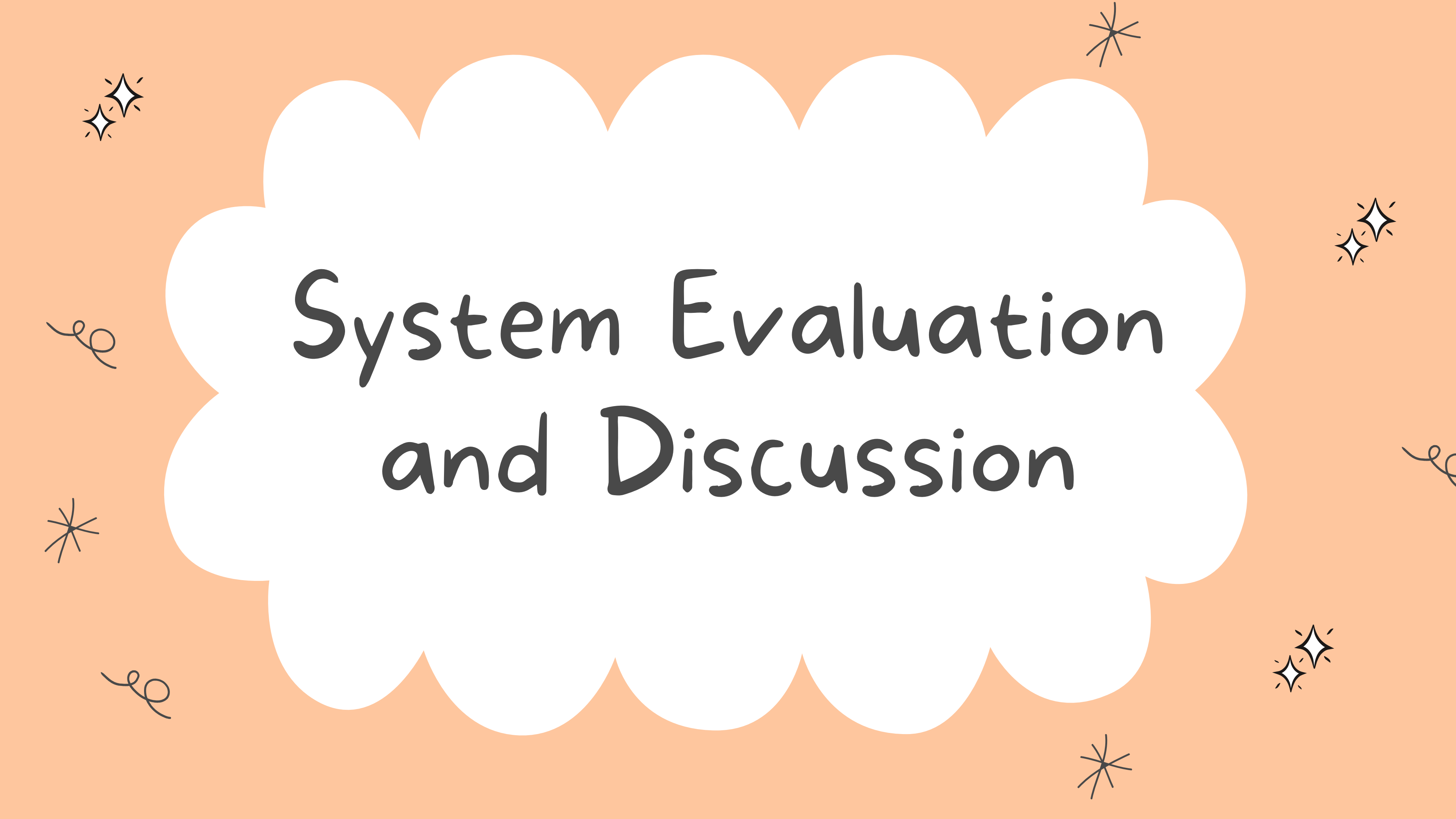
# Function to send a message to a specific Telegram chat
def send_telegram_message(message):
    url = f"https://api.telegram.org/bot{BOT_TOKEN}/sendMessage"
    params = {
        "chat_id": CHAT_ID,
        "text": message
    }

    try:
        response = requests.get(url, params=params, timeout=60)
        if response.status_code == 200:
            print("Message sent successfully!")
        else:
            print(f"Failed to send message: {response.text}")
    except requests.exceptions.Timeout:
        print("The request timed out.")
    except Exception as e:
        print(f"Error occurred: {e}")

# Function to check attendance and send messages if below 80%
def attendance_tracking(student_name, course_name, attendance_percentage):
    if attendance_percentage < 80:
        message_body = f"Hi {student_name}, your {course_name} attendance is bel
        send_telegram_message(message_body)

# Load attendance data from the JSON file
```





System Evaluation and Discussion

System Evaluation and Discussion

20

- **Performance Metrics:** response time, scalability, resource consumption, and user experience
- **Testing Result:** .PIR trigger with motion, ultrasonic measure distance, blink detection update attendance, check attendance via student id, telegram bot send alert message
- **Project Challenges:** Lighting issues, limited processing power, sensor calibration problems, and privacy concerns.
- **Objectives Evaluation:** Automated attendance, reduced human error, provided real-time monitoring, scalability in large area.



Conclusion



- This project builds a low-cost, real-time, contactless attendance system.
- It uses simple hardware and open-source tools for accessibility.
- Face recognition accuracy may be affected by lighting and occlusion.
- Motion detection may require precise calibration
- The system improves efficiency in real-time attendance tracking and reduces manual work.
- With further refinement, it can be deployed in a variety of small-scale environments such as offices or classrooms.

Thank you
Q&A

